

Teste técnico prático: *avaliação de desempenho*

Duração estimada: 3 a 5 horas

Formato: aplicação rodando localmente + entrega de artefatos

Prazo de entrega: 3 dias corridos após recebimento

SOBRE ESTE TESTE

Diferente de um teste puramente teórico, aqui você recebe uma **aplicação real e funcional**: o módulo de Avaliação de Desempenho de um produto SaaS de RH. Você vai subir o sistema na sua máquina, explorá-lo como um usuário e como um QA, e produzir os artefatos de qualidade que garantiriam a entrega segura desse módulo.

O entregável inclui quatro frentes: (1) uma **estratégia de teste com estimativa de esforço**, (2) **casos de teste** estruturados, (3) um **plano de testes de API**, e (4) uma **suíte de automação** escrita dentro do próprio projeto, na ferramenta que você preferir.

Não existe resposta única correta. Avaliamos a qualidade do raciocínio, a cobertura e a profundidade analítica — e, na automação, a estrutura, a clareza e a facilidade de execução do que você entregar.

⚠ Comece pela Seção 01 (suba a aplicação) e explore o sistema antes de escrever qualquer artefato. Vários comportamentos só ficam evidentes usando o produto.

01 A APLICAÇÃO E COMO EXECUTÁ-LA

Você recebeu o repositório do módulo de **Avaliação de Desempenho**. É uma aplicação web completa (API + interface) que sobe localmente, sem dependências externas. O banco é **SQLite em memória**: é criado e populado a cada inicialização e **zerado** ao desligar o servidor — ou seja, reiniciar devolve os dados ao estado inicial.

Stack — Backend em Node.js (≥ 20) + Express; banco SQLite em memória; frontend em React + Vite. Tudo sobe com um único comando.

```
# Pré-requisito: Node.js 20+ (verifique com: node -v)
npm install          # instala backend + frontend
npm run dev          # sobe a API (porta 3001) e a interface (porta 5173)

# Acesse a interface: http://localhost:5173
# Base da API:        http://localhost:3001/api
```

Consulte o **README.md** do projeto para detalhes e solução de problemas. Não há login: o "usuário atual" é escolhido por um seletor na tela de respostas (impersonação) — uma simplificação intencional para facilitar o teste.

O sistema cobre as seguintes entidades e regras:

Competências — o que é avaliado (apenas um nome). São vinculadas aos ciclos.

Times — organograma hierárquico (um time pode ter um time superior, sem limite de profundidade).

Colaboradores — os avaliados. Cada um pertence a exatamente um time e pode ter zero ou um gestor direto (que também é um colaborador).

Ciclos — período + regras de uma rodada: nome, datas, times participantes e competências. Têm a opção de *pesos por competência*: quando ativa, a soma dos pesos deve ser exatamente **100**. Configuração só é editável enquanto o ciclo está em *rascunho (draft)*.

Avaliações — geradas automaticamente ao iniciar o ciclo. Cada colaborador dos times participantes recebe: (1) uma **autoavaliação**, (2) uma **avaliação do gestor** direto (se houver) e (3) uma **avaliação de cada colega do mesmo time**. São respondidas (notas de 1 a 5 por competência) somente enquanto a data atual estiver dentro do período. Ao final, o sistema calcula médias e emite relatórios.

Antes de testar, planeje. Esta seção avalia como você organiza o trabalho de QA sobre um sistema real, com tempo limitado.

2.1 ESTRATÉGIA E ESCOPO

Depois de explorar a aplicação, descreva sua **estratégia de teste**: o que será testado, em quais níveis (interface e API), quais tipos de teste aplicará (funcional, negativo, borda, fluxo de ponta a ponta) e o que ficará *fora de escopo* — justificando. Aponte também as **ambiguidades ou riscos** que você levantaria com o time de produto antes de começar (mínimo 5).

Dica: pense em estados e transições (draft → active → closed), regras implícitas e cenários de borda — não só no fluxo feliz.

ENTREGÁVEL

Documento com a estratégia, matriz de escopo (in/out) e lista de riscos/ambiguidades com a pergunta que você faria ao produto.

2.2 ESTIMATIVA DE ESFORÇO

Estime o esforço de QA para homologar este módulo, **quebrado por área** (cadastros, configuração de ciclos, geração de avaliações, resposta/período, relatórios/médias, automação). Apresente em **tabela** com a estimativa em horas, a técnica que usou para estimar e as premissas assumidas. Indique o total e como ele caberia (ou não) em uma sprint de 10 dias úteis com 50% do seu tempo dedicado.

ENTREGÁVEL

Tabela de estimativa por área (horas + premissas) e uma breve conclusão sobre viabilidade no prazo.

2.3 PRIORIZAÇÃO

Se o tempo não fosse suficiente para tudo, em que ordem você testaria? Liste as áreas por prioridade (alta / média / baixa) e justifique com base em risco e impacto para o negócio.

ENTREGÁVEL

Lista priorizada com justificativa de risco/impacto por item.

CENÁRIO JÁ CARREGADO NA APLICAÇÃO (SEED) — USE COMO BASE

Ao subir o sistema, ele já vem populado com a empresa **Acme Corp**:

Times: Tecnologia (raiz) → Back-end e Front-end (filhos).

Back-end: Ana (sem gestor), Bruno (gestor: Ana), Carlos (gestor: Ana).

Front-end: Diana (sem gestor), Eduardo (gestor: Diana).

Competências: Comunicação, Entrega, Colaboração.

Ciclo: "Avaliação de Desempenho — Ciclo Atual", sem pesos, com Back-end e Front-end e as 3 competências, em status *rascunho* (*draft*). O período é relativo a hoje, então, ao iniciar o ciclo, as respostas já ficam liberadas.

Escreva casos de teste para as funcionalidades abaixo. Para cada caso informe: **título, pré-condição, passos, resultado esperado e classificação** (positivo / negativo / borda). Não há formato fixo — use tabela, texto estruturado ou BDD.

3.1 GERAÇÃO DE AVALIAÇÕES AO INICIAR O CICLO

Inicie o ciclo do seed e verifique quais avaliações foram geradas para cada colaborador (autoavaliação, gestor e pares). Escreva os casos que comprovam a regra — incluindo bordas como colaborador *sem gestor*, colaborador *sozinho no time* e a relação entre gestor e par.

Mínimo: 8 casos. Profundidade vale mais que volume.

ENTREGÁVEL

Conjunto de casos de teste para a geração, cobrindo positivos, negativos e bordas.

3.2 RESPOSTA DE AVALIAÇÃO E CONTROLE DE PERÍODO

Cubra a regra "só é possível responder com a data atual dentro do período do ciclo", além das validações de resposta: notas de 1 a 5, todas as competências obrigatórias, avaliador correto e reenvio de avaliação já respondida. Para o período, considere os estados: antes do início, durante, no último dia, após o fim e ciclo de data única (início = fim).

ENTREGÁVEL

Casos de teste de resposta + matriz de estados de período (com resultado esperado por estado).

3.3 CONFIGURAÇÃO DE CICLO E PESOS

Escreva casos para a criação/edição de ciclos: datas (início $\leq$ fim, datas iguais válidas), exigência de ao menos 1 time e 1 competência, edição permitida apenas em rascunho, e a regra de pesos (quando ativos, a soma deve ser exatamente 100). Inclua os casos negativos correspondentes.

ENTREGÁVEL

Casos de teste de configuração de ciclo, com foco nas validações e na regra de pesos.

3.4 RELATÓRIOS E CÁLCULO DE MÉDIAS

Responda algumas avaliações e valide os relatórios (analítico, sintético e de resultado). Escreva casos para o cálculo de médias por competência e por tipo de avaliador, incluindo: colaborador sem gestor, colaborador único no time e ciclo *parcialmente respondido* (nem todos avaliaram). Se quiser, valide também o efeito dos pesos na média final.

ENTREGÁVEL

Casos de teste de relatórios/médias, com os valores esperados calculados manualmente para comparação.

04 TESTES DE API (POSTMAN, INSOMNIA OU SIMILAR)

OBRIGATÓRIO

Abaixo estão os **contratos reais** de dois endpoints centrais — exatamente como a aplicação responde. Todas as rotas ficam sob a base `http://localhost:3001/api` e os erros seguem o formato `{ "error": "codigo", "details": [...] }`. Como não há autenticação, o `evaluator_id` é enviado no corpo da resposta.

POST /api/cycles/:id/start

// Inicia o ciclo (draft → active) e gera as avaliações. Sem body.

// 200 – sucesso

```
{
  "cycle_id": 1,
  "status": "active",
  "evaluations_generated": 13,
  "started_at": "2026-06-15"
}
```

// 409 cycle_already_started – ciclo não está em draft

// 422 invalid_cycle_config – sem time/competência, ou pesos ≠ 100

// 404 cycle_not_found – id inexistente

POST /api/evaluations/:id/answer

// Registra as respostas e conclui a avaliação.

// Request body

```
{
  "evaluator_id": 2,
  "answers": [
    { "competency_id": 1, "score": 4 },
    { "competency_id": 2, "score": 3 },
    { "competency_id": 3, "score": 5 }
  ]
}
```

// 200 – { "evaluation_id": 101, "status": "completed" }

// 400 invalid_score – nota fora de 1..5

// 403 forbidden – evaluator_id não corresponde ao da avaliação

// 409 already_answered – avaliação já respondida

// 410 cycle_expired – fora do período do ciclo

// 422 missing_competencies – falta resposta de alguma competência do ciclo

Demais rotas disponíveis: /competencies, /teams, /employees, /cycles, /cycles/:id/close, /evaluations (com filtros cycle_id, evaluator_id, evaluated_id, status) e /reports/... . Explore-as à vontade.

4.1 COLEÇÃO DO FLUXO COMPLETO

Monte uma coleção (Postman/Insomnia/Bruno/etc.) que percorra o fluxo de ponta a ponta: **criar e configurar um ciclo** → **iniciar** → **listar avaliações** → **responder** → **consultar relatórios**. Use variáveis para encadear os IDs entre as requisições. Exporte a coleção e inclua na entrega.

ENTREGÁVEL

Arquivo da coleção exportado (JSON) + breve descrição do fluxo.

4.2 PLANO DE TESTES — INICIAR CICLO E RESPONDER AVALIAÇÃO

Para os dois endpoints acima, descreva os testes que executaria. Para cada um informe: objetivo, dados de entrada, e validações na resposta (status code, corpo, consistência dos dados). Vá além do status code — valide o **contrato**, a **consistência** (ex.: o número de avaliações geradas bate com a regra de negócio?) e os cenários de erro listados nos contratos.

ENTREGÁVEL

Plano de testes de API por endpoint, com os cenários de sucesso e de erro.

4.3 BUG REPORT

Durante a exploração (interface ou API), registre **pelo menos um bug ou inconsistência** que encontrar, no formato que usaria no Jira: título, severidade/prioridade, ambiente, passos para reproduzir, resultado obtido, resultado esperado e evidência. Se não encontrar um defeito claro, documente um **comportamento ambíguo** (que poderia ser bug ou intencional) e qual seria sua recomendação ao produto.

ENTREGÁVEL

Um ou mais bug reports completos (ou ambiguidades documentadas com recomendação).

05 AUTOMAÇÃO DE TESTES

OBRIGATÓRIO

Escreva uma **suíte de testes automatizados dentro do próprio projeto**, na ferramenta de sua preferência. Você escolhe o nível e o stack:

- ☐ API: Jest + Supertest, Postman/Newman, Pytest + requests, RestAssured, k6, etc.

- Interface (E2E): Playwright, Cypress, Selenium, etc.
- Vale combinar os dois níveis — explique sua escolha e o porquê.

5.1 SUÍTE AUTOMATIZADA DOS FLUXOS CRÍTICOS

Automatize, no mínimo, os cenários críticos do módulo. Sugestão de cobertura (adapte conforme sua estratégia):

- Geração: ao iniciar o ciclo do seed, Bruno recebe 1 autoavaliação + 1 de gestor + 1 de par; Ana, 1 + 0 + 2; Eduardo, 1 + 1 + 0.
- Resposta (feliz): responder uma avaliação dentro do prazo retorna `200` e `completed`.
- Resposta (erros): avaliador errado → `403`; nota inválida → `400`; competência faltando → `422`; reenvio → `409`; ciclo no passado → `410`.
- Pesos: iniciar ciclo com pesos somando $\neq 100$ → `422 invalid_cycle_config`.
- Relatório: validar uma média calculada contra o valor esperado.

Mínimo: 6 a 10 verificações automatizadas. Qualidade e organização contam mais que quantidade.

ENTREGÁVEL

Código da automação em uma pasta própria no repositório (ex.: `/qa-automation` ou `/tests`).

5.2 EXECUÇÃO E DOCUMENTAÇÃO

A suíte deve rodar com **um comando documentado** e ter um README curto explicando: ferramenta escolhida e por quê, como instalar e executar, e o que cada grupo de testes cobre.

Atenção a um ponto-chave: o banco é em memória e reinicia a cada start do servidor — descreva como sua automação lida com o estado (ex.: subir o servidor limpo antes da suíte, ou tornar os testes independentes da ordem de execução).

ENTREGÁVEL

README da automação + comando único de execução + estratégia de gestão de estado/dados.

06 DIFERENCIAIS (OPCIONAL)

BÔNUS

Itens opcionais que valem como diferencial. Faça apenas se sobrar tempo — não comprometa o obrigatório.

- ☐ CI: configure um workflow (ex.: GitHub Actions) que suba a aplicação e rode sua suíte de automação a cada push.
- ☐ Cobertura adicional: testes de regressão da hierarquia de times/gestão, geração de massa de dados ou cenários de carga simples.
- ☐ Relatório de execução: gere um relatório (HTML/JUnit) dos testes automatizados e inclua na entrega.
- ☐ Análise crítica: documente um cenário complexo de dados inconsistentes (ex.: colaborador desligado no meio de um ciclo ativo) e o comportamento esperado que você proporia ao produto.

FORMATO DE ENTREGA

Entregue um **repositório Git** (link público/privado com acesso) ou um **.zip** contendo: (1) a aplicação fornecida; (2) a **automação** em uma pasta própria, com README e comando de execução; (3) os **documentos de QA** (estratégia, estimativa, casos de teste, plano de API e bug reports) em PDF, Word ou Markdown; (4) a **coleção de API** exportada.

Não há formato fixo para os casos de teste — use o que for mais natural (tabela, texto estruturado, BDD). O que importa é que o **raciocínio fique explícito**: mostre como você pensou, não apenas o resultado. Documente as ferramentas que escolheu e o porquê.